

SISTEMAS DISTRIBUÍDOS TALLER 6

PROYECTO FINAL - PARTE III

OBJETIVO: Desplegar la tercera parte del proyecto final de curso, evaluando conocimientos técnicos en manejo de sesiones, HTTP y WS.

Hasta ahora tenemos un solo coordinador. Toda la lógica del juego, el estado, las conexiones vive en un solo proceso. Si ese proceso se cae, se acabó el juego para todos. Y si muchos jugadores se conectan, el cuello de botella es el mismo proceso aguantando todas las conexiones WS.

Este taller resuelve eso:

- Múltiples coordinadores en paralelo, todos con el mismo estado del juego, intercambiables.
- Cada cliente se conecta al coordinador menos cargado. Si hay 100 jugadores y 4 coordinadores, ~25 jugadores van a cada uno.
- Los coordinadores se replican entre sí. Si Alice está conectada al coordinador A y se mueve, Bob (conectado al coordinador B) la ve moverse.
- El cliente muestra a qué coordinador está conectado.

No es trivial. Hay varios problemas interesantes que van a tener que resolver: descubrimiento de pares, evitar bucles infinitos de replicación, mantener consistencia, manejar coordinadores que entran y salen del mesh.

Vamos a hacer tres cambios grandes respecto al Taller 2:

- El auth service ahora también es un directory service. Maneja un endpoint donde los coordinadores se registran con heartbeats, y un endpoint donde los clientes preguntan a cuál conectarse.
- Los coordinadores forman un mesh entre sí vía WebSocket. Cada coordinador se conecta a todos los demás.
- El cliente hace dos pasos antes de jugar: login → pedir coordinador → conectar WS al coordinador asignado.

1. Lo qué se va a construir

Paso 1: Endpoints nuevos (HTTP)

POST /heartbeat

Lo llama cada coordinador cada 2 segundos para anunciar que sigue vivo.

Request:

```
JSON
{
  "coordinatorId": "coord-a",
  "publicUrl": "ws://localhost:5001",
  "peerUrl": "ws://localhost:5101",
  "connectedPlayers": 7,
  "uptime": 12345
}
```

- **coordinatorId**: Identificador único del coordinador. Lo elige el coordinador al arrancar (ej: lo lee de su `.env`).
- **publicUrl**: La URL pública a la que se conectan los clientes (los jugadores).
- **peerUrl**: La URL a la que se conectan los otros coordinadores (para el mesh). Puede ser el mismo puerto u otro. Recomendado: Puerto distinto para separar tráfico de clientes y de peers.
- **connectedPlayers**: Cuántos clientes tiene conectados localmente.
- **uptime**: En segundos. Útil para tu UI de debug.

Response: 200 { "ok": true }.

El auth-service guarda en memoria un `Map<coordinatorId, { ...info, lastSeen: Date.now() }>`. Cualquier coordinador que no haya hecho heartbeat en los últimos 6 segundos (3 ciclos) se considera muerto y se descarta.

GET /coordinator

Lo llama el cliente después de hacer login, antes de abrir el WS.

Response:

```
JSON
{
  "coordinatorId": "coord-b",
  "publicUrl": "ws://localhost:5002"
}
```

Estrategia de selección: El auth devuelve el coordinador con menos `connectedPlayers` entre los que estén vivos. Si hay empate, el primero que encuentre. Si no hay ninguno vivo, responde `503 { "error": "no_coordinators_available" }`.

Esto es load balancing del lado del cliente (client-side load balancing). El balanceador no proxea tráfico, solo le dice al cliente a dónde conectarse. Es lo que hace Discord, lo que hace cualquier sistema serio.

GET /peers

Lo llaman los coordinadores al arrancar (y periódicamente) para descubrir a sus pares en el mesh.

Response:

```
JSON
{
  "peers": [
    { "coordinatorId": "coord-a", "peerUrl": "ws://localhost:5101" },
    { "coordinatorId": "coord-c", "peerUrl": "ws://localhost:5103" }
  ]
}
```

(Cada coordinador filtra su propio ID de la lista que recibe, obviamente.)

Paso 2: Protocolo del mesh

Cada coordinador escucha WS en su **peerUrl** (separado del WS de clientes). Cuando descubre un nuevo peer en **/peers**, abre una conexión WS hacia él.

Importante: Cada par de coordinadores tiene una sola conexión WS, no dos. Si A descubre a B y B descubre a A simultáneamente, alguien va a tener que cerrar la duplicada. La forma estándar: el que tiene el **coordinatorId** "menor" (orden lexicográfico) es el que inicia la conexión; el otro espera a recibirla. Si te llega una conexión de un peer con ID "mayor" al tuyo y tu ya tienes una hacia él, cierras la nueva.

Todos los mensajes llevan un campo **origin** con el **coordinatorId** que originó el mensaje. Eso es lo que te permite evitar bucles: si recibes un mensaje cuyo **origin** seas tú mismo, lo descartas.

Handshake (primer mensaje al abrir la conexión):

JSON

```
{ "type": "hello", "coordinatorId": "coord-a" }
```

Replicación de intent (cuando un cliente local manda un intent):

JSON

```
{  
  "type": "intent_replicate",  
  "origin": "coord-a",  
  "userId": 42,  
  "intent": { "dir": { "x": 1, "y": 0 } }  
}
```

El coordinador que recibe este mensaje aplica el intent a **su copia local** del jugador 42 (que puede o no estar conectado localmente — el coord lo tiene en el estado igual). No lo retransmite — los demás coordinadores también lo reciben directamente del origen.

Replicación de extras (cuando un cliente local manda extras_update):

JSON

```
{  
  "type": "extras_replicate",  
  "origin": "coord-a",  
  "userId": 42,
```

```
"extras": { "color": "#ff5500" }  
}
```

Replicación de conexión / desconexión (cuando un cliente se conecta o desconecta a un coordinador):

JSON

```
{  
  "type": "player_joined",  
  "origin": "coord-a",  
  "userId": 42,  
  "username": "alice",  
  "x": 100,  
  "y": 200  
}  
  
{  
  "type": "player_left",  
  "origin": "coord-a",  
  "userId": 42  
}
```

Tu game loop a 20 Hz ahora tiene que:

1. Aplicar los intents de todos los jugadores (locales y remotos) a sus posiciones.
2. Broadcast del state solo a sus clientes locales (NO a peers).

Cada coordinador hace su propio broadcast a sus clientes. Cada cliente solo recibe state de su coordinador, pero ese state incluye a todos los jugadores del mundo.

Paso 3: Cambios en el cliente

1. Antes de abrir el WS al coordinador, el cliente hace **GET /coordinator** al auth-service. Recibe { **coordinatorId**, **publicUrl** }.
2. Abre el WS a ese **publicUrl** con el token (como antes).
3. Muestra en la UI a qué coordinador está conectado. Donde está "Tick: 20 Hz" agregas "Coordinador: coord-b" o algo similar. Hazlo visible.
4. Si el WS se cae, NO reconectar directo al mismo coordinador. Volver a pedir **GET /coordinator** y conectarse al que diga.

2. Entrega y sustentación

Qué se entrega:

1. **Link al repo de GitHub.** Commits incrementales, no uno solo.
2. **README completo.**
3. **Sistema corriendo con túneles en ngrok durante la sustentación.**

Qué se sustenta:

1. Levantas tres coordinadores con distintos puertos. Los tres aparecen en `/peers`. (3 min).
2. Conecta 4 clientes y muestras que el auth los distribuye razonablemente entre los tres. (No tiene que ser exactamente balanceado, pero no deberían ir los 4 al mismo). (3 min).
3. Te conectas como Alice a coord-A y como Bob a coord-B en pestañas distintas. Muestras en cada pestaña a qué coordinador está conectada. Mueves a Alice y Bob la ve moverse en vivo. Mueves a Bob y Alice lo ve también en vivo. (2 min).
4. Matás un coordinador con Ctrl+C. Muestras que:
 - a. Los clientes que estaban en ese coordinador se desconectan.
 - b. Cuando esos clientes vuelven a pedir `/coordinator`, el auth ya no devuelve el muerto (porque pasaron > 6 segundos sin heartbeat).
 - c. Los clientes se reconectan a los coordinadores vivos (4 min).
5. Responde preguntas (3 min).